Practical - 8

Practical Title: MySQL Schema Design, ER Model, PDO Connectivity, and Prepared Statements

Problem Definition: Design the database for StudentHub with students, events and registrations. Create PHP database connection using MYSQL statements.

Theory

1. Is the schema normalized properly?

Answer: Yes, the StudentHub database should be designed using normalization so that data is organized into separate related tables and unnecessary duplication is avoided.
For example, instead of storing student details repeatedly in a registration table, we can create separate tables such as:

- students – stores student information.
- events – stores event information.
- registrations – stores which student registered for which event.

Normalization helps reduce data redundancy and makes the database easier to maintain.
W3Schools explains that MySQL tables can use constraints such as PRIMARY KEY and FOREIGN KEY to maintain reliable relationships between tables.

Reference Link:
https://www.w3schools.com/mysql/mysql_constraints.asp
https://www.w3schools.com/php/php_mysql_create_table.asp

2. Are primary keys and foreign keys correctly defined?

Answer: Yes. Each main table should have a primary key that uniquely identifies every record.
For example:

```
CREATE TABLE students (
   student_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE,
   course VARCHAR(100)
);
```

Here, student_id is the primary key.
The registrations table can use foreign keys:

```
CREATE TABLE registrations (
   registration_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    student_id INT,
    event_id INT,
    registration_date DATE,

    FOREIGN KEY (student_id)
        REFERENCES students(student_id),

    FOREIGN KEY (event_id)
        REFERENCES events(event_id)
);
```

A primary key uniquely identifies a record and cannot contain NULL values. A foreign key creates a relationship between a child table and a parent table and helps prevent invalid references.

Reference Link:

https://www.w3schools.com/mysql/mysql_primarykey.asp
https://www.w3schools.com/mysql/mysql_foreignkey.asp

3. Is the database connection handled securely?

Answer: Yes, the database connection should be handled using PDO with proper exception handling. The database credentials should not be repeatedly written throughout the application; instead, a separate file such as db.php can be used.

Example:

```
<?php

$host = "localhost";
$dbname = "studenthub";
$username = "root";
$password = "";

try {
    $conn = new PDO(
        "mysql:host=$host;dbname=$dbname",
        $username,
        $password
    );

    $conn->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
```

```
    );

    echo "Database connected successfully";

} catch (PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>
```

PDO supports MySQL connections and can use exception-based error handling. W3Schools also notes that prepared statements help protect against SQL injection by separating SQL commands from the data being supplied.

For example:

```
$sql = "INSERT INTO students (name, email, course)
        VALUES (?, ?, ?)";

$stmt = $conn->prepare($sql);

$stmt->execute([
    $name,
    $email,
    $course
]);
```

Reference Link:

https://www.w3schools.com/php/php_mysql_connect.asp

https://www.w3schools.com/php/php_mysql_prepared_statements.asp

A. Implementation/ Output/Result

students.php

```
<?php
require_once "db.php";

$sql = "SELECT * FROM students";
$result = mysqli_query($conn, $sql);
?>

<!DOCTYPE html>
```

```
<html>
<head>
    <title>Students</title>
</head>

<body>

<h1>Student List</h1>

<table border="1" cellpadding="10">

<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Gender</th>
</tr>

<?php

while ($row = mysqli_fetch_assoc($result)) {

    echo "<tr>";

    echo "<td>" . htmlspecialchars($row['id']) . "</td>";
    echo "<td>" . htmlspecialchars($row['name']) . "</td>";
    echo "<td>" . htmlspecialchars($row['email']) . "</td>";
    echo "<td>" . htmlspecialchars($row['gender']) . "</td>";

    echo "</tr>";
}

?>

</table>

</body>
</html>
```

events.php

<?php

```
require_once "db.php";

$sql = "SELECT * FROM events";
$result = mysqli_query($conn, $sql);
?>

<!DOCTYPE html>
<html>
<head>
   <title>StudentHub Events</title>
</head>

<body>

<h1>StudentHub Events</h1>

<?php
if (mysqli_num_rows($result) > 0) {

   while ($row = mysqli_fetch_assoc($result)) {
      echo "<h2>" . htmlspecialchars($row['event_name']) . "</h2>";
      echo "<p>Date: " . htmlspecialchars($row['event_date']) . "</p>";
      echo "<p>" . htmlspecialchars($row['description']) . "</p>";
      echo "<hr>";
   }
```

```
} else {
    echo "<p>No events available.</p>";
}
?>

</body>
</html>
```

registration.php

```
<?php

require_once "db.php";

// Check whether form was submitted using POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // Get form values
    $fullname = trim($_POST["fullname"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $mobile = trim($_POST["mobile"] ?? "");
    $course = trim($_POST["course"] ?? "");
    $year = trim($_POST["year"] ?? "");
    $gender = trim($_POST["gender"] ?? "");
    $password = $_POST["password"] ?? "";
    $confirm_password = $_POST["confirm_password"] ?? "";
```

```
$terms = $_POST["terms"] ?? "";

// Validation
$errors = [];

// Full name
if (!preg_match("/^[A-Za-z\s]{3,}$/", $fullname)) {
    $errors[] = "Please enter a valid full name.";
}

// Email
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Please enter a valid email.";
}

// Mobile
if (!preg_match("/^[6-9][0-9]{9}$/", $mobile)) {
    $errors[] = "Please enter a valid 10-digit mobile number.";
}

// Course
if ($course == "") {
    $errors[] = "Please select a course.";
}
```

```
// Year
if ($year == "") {
    $errors[] = "Please select your year.";
}

// Gender
if ($gender == "") {
    $errors[] = "Please select your gender.";
}

// Password
if (!preg_match(
    "/^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@$!%*?&]).{8,}$/",
    $password
)) {
    $errors[] =
    "Password must contain uppercase, lowercase, number and special character.";
}

// Confirm password
if ($password != $confirm_password) {
    $errors[] = "Passwords do not match.";
}

// Terms
```

```
if ($terms != "accepted") {
    $errors[] = "You must accept the Terms and Conditions.";
}

// If there are validation errors
if (!empty($errors)) {

    echo "<h2>Registration Failed</h2>";
    echo "<ul>";

    foreach ($errors as $error) {
        echo "<li>" . htmlspecialchars($error) . "</li>";
    }

    echo "</ul>";

    echo '<a href="signup.php">Go Back</a>';

    exit;
}

// Hash password before storing it
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);

// Insert into students table
```

```
$sql = "INSERT INTO students
        (name, email, password, gender, mobile, course, year)
        VALUES (?, ?, ?, ?, ?, ?, ?)";

$stmt = mysqli_prepare($conn, $sql);

if (!$stmt) {
    die("Database error: " . mysqli_error($conn));
}

mysqli_stmt_bind_param(
    $stmt,
    "ssssssi",
    $fullname,
    $email,
    $hashedPassword,
    $gender,
    $mobile,
    $course,
    $year
);

// Execute query
if (mysqli_stmt_execute($stmt)) {
```

```
    echo "<h2>Registration Successful!</h2>";

    echo "<p>Welcome to StudentHub, "
        . htmlspecialchars($fullname)
        . "!</p>";

    echo "<p>Your registration has been saved successfully.</p>";

    echo '<a href="signup.php">Go Back to Sign Up</a>';

} else {

    // Duplicate email
    if (mysqli_errno($conn) == 1062) {

        echo "<h2>Email Already Registered</h2>";
        echo "<p>This email address is already registered.</p>";

    } else {

        echo "<h2>Registration Failed</h2>";
        echo "<p>"
            . htmlspecialchars(mysqli_error($conn))
            . "</p>";
    }
```

```
    }

    mysqli_stmt_close($stmt);
    mysqli_close($conn);

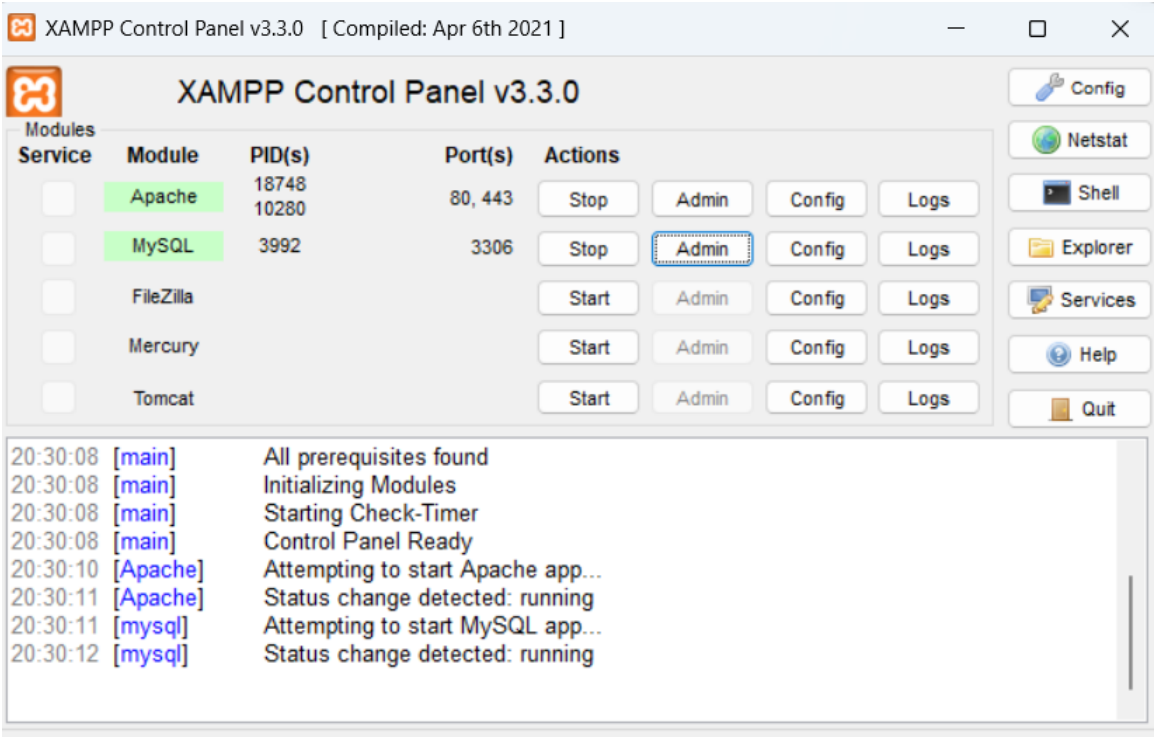
} else {

    echo "Invalid Request.";

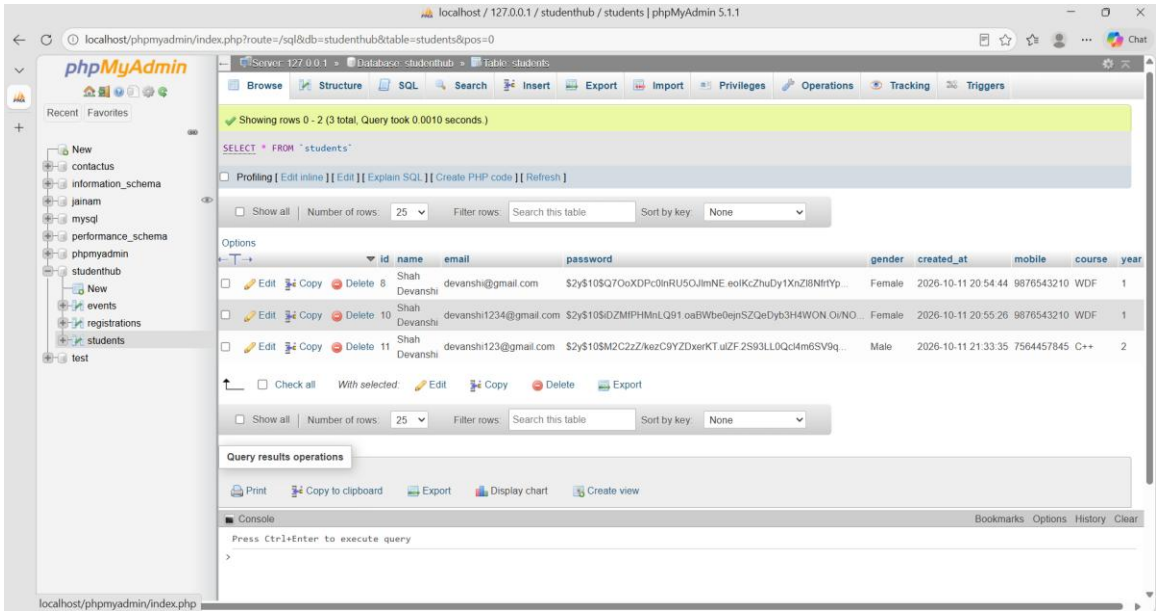
}

?>
```

B. Evidence of Work (Screenshots)



Description: This screenshot shows the XAMPP Control Panel v3.3.0, where both Apache and MySQL are running successfully. Apache is running on ports 80 and 443, while MySQL is running on port 3306. Apache is responsible for handling web requests, and MySQL manages the database required for the StudentHub web application. The status messages confirm that both services have started successfully, indicating that the local server environment is ready for development and testing.



Description: This screenshot shows phpMyAdmin, a web-based database management tool, displaying the studenthub database and its students table. The table contains three student records with details such as ID, name, email, password, gender, creation date, mobile number, course, and year. The interface provides various options, including Browse, Structure, SQL, Search, and Insert, to view and manage database records. The navigation panel also displays the events and registrations tables. Overall, this screenshot demonstrates that the StudentHub database has been created and student information is stored in the database.

db.php U

Practical8 > db.php > ...

```php
<?php

$host = "localhost";
$username = "root";
$password = "";
$database = "studenthub";

$conn = mysqli_connect($host, $username, $password, $database);

if (!$conn) {
    die("Database connection failed: " . mysqli_connect_error());
}

?>
```

Description: The db.php file is used to establish a connection between a PHP application and a MySQL database named studenthub. It defines the database connection details, including the host (localhost), username (root), password (empty in this example), and database name. The mysqli_connect() function uses these details to connect to the database, and the connection is stored in the $conn variable. The if (!$conn) condition checks whether the connection has failed. If the connection is unsuccessful, the die() function stops the script and displays an error message using mysqli_connect_error(). This file allows other PHP pages to reuse the database connection and perform operations such as inserting, retrieving, updating, and deleting student records.

C. GitHub Repository Link:

https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical8

D. Challenges Faced:

Designing the database schema

Initially, it was difficult to decide which information should be stored in each table. Separating students, events and registrations helped avoid duplicate data.

Defining primary and foreign keys

Understanding the relationship between `student_id`, `event_id` and `registration_id` was challenging. Proper primary and foreign keys were required to maintain relationships between tables.

Creating the database using phpMyAdmin

Creating databases and tables and entering SQL queries through phpMyAdmin required careful attention to table names, column names and data types.

Connecting PHP with MySQL

Configuring the correct hostname, database name, username and password in the PDO connection was initially challenging.

Handling database errors

Connection errors, incorrect table names and SQL syntax errors had to be identified and handled using PDO exceptions.

Understanding prepared statements

It was initially difficult to understand why placeholders such as `?` are used instead of directly inserting PHP variables into SQL queries. Prepared statements separate SQL commands from supplied values and help protect against SQL injection.

E. Learning Outcomes:

- How to design a relational database schema for a web application.
- How to create an ER model and identify relationships between entities.
- How to create databases and tables using MySQL and phpMyAdmin.
- How to define primary keys and foreign keys.
- How to normalize database tables and reduce duplicate data.
- How to connect a PHP application with MySQL using PDO.
- How to use PDO exception handling for database errors.
- How to use prepared statements for safer database operations.
- How to insert and retrieve records from MySQL using PHP.
- How a database can be integrated into a StudentHub web application.

Faculty Remarks: (Keep this blank for faculty)